FACULTAD DE INFORMÁTICA

Universidad de Murcia

Programación Concurrente y Distribuida

Memoria de Prácticas

Realizado por:

García Illán, Juan César

Jiménez León, Sergio

Profesor:

José Miguel Bolarín Guillén

Grupo y Subgrupo de Prácticas:

Grupo 3 — Subgrupo 3

Convocatoria: Ordinaria (Junio)

Año Académico: 2025–2026

Índice

1. Ejercicio 1 — ReentrantLock **2**
1.1. Recursos no compartibles 2
1.2. Condiciones de sincronización 2
1.3. Pseudocódigo comentado 2
1.4. Cuestiones 5
1.5. Código Java 5

2. Ejercicio 2 — Semáforos **10**
2.1. Recursos no compartibles 10
2.2. Condiciones de sincronización 10
2.3. Pseudocódigo comentado 10
2.4. Cuestiones 12
2.5. Código Java 12

3. Ejercicio 3 — Monitores **17**
3.1. Recursos no compartibles 17
3.2. Condiciones de sincronización 17
3.3. Pseudocódigo comentado 17
3.4. Cuestiones 21
3.5. Código Java 22

4. Ejercicio 4 — Paso de Mensajes Asíncrono **32**
4.1. Recursos no compartibles 32
4.2. Condiciones de sincronización 32
4.3. Pseudocódigo comentado 32
4.4. Cuestiones 34
4.5. Código Java 34

1. Ejercicio 1 — ReentrantLock

1.1. Recursos no compartibles

En este ejercicio, cada hilo trabaja con sus propias matrices locales, así que no comparten datos entre ellos. El único recurso no compartible es:

- **La pantalla** (`System.out`): Imprimir la matriz completa requiere varias llamadas a `println` seguidas. Si los dos hilos imprimen a la vez, sus líneas se mezclan y la salida es ilegible. La pantalla es un recurso hardware que requiere acceso exclusivo para escritura.

1.2. Condiciones de sincronización

En este problema **no hay condiciones de sincronización**. Según la teoría, una condición de sincronización es una relación de precedencia entre procesos: un proceso tiene que esperar a que otro haga algo concreto antes de poder continuar. Aquí ningún hilo depende del resultado del otro para hacer su trabajo; cada uno genera su propia matriz y la imprime de forma totalmente independiente.

Lo que sí existe es **exclusión mutua** sobre la pantalla. El bloque de impresión de cada hilo es una sección crítica, y se protege con un `ReentrantLock` compartido (`micerrojo`). Cada hilo hace `lock()` antes de empezar a imprimir y `unlock()` al terminar, garantizando que ningún otro hilo pueda imprimir a la vez.

1.3. Pseudocódigo comentado

Listing 1: Pseudocódigo HiloMult

```
HiloMult(micerrojo)

    for (int reps = 0; reps < 10; reps++) {
        int[][] matriz = new int[3][3];
        int[][] matrizRes = new int[3][3];

        // Rellenar matriz aleatoria
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                matriz[i][j] = rand.nextInt(100);
            }
        }

        // Hacemos la multiplicacion
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                matrizRes[i][j] = matriz[i][0]*matriz[0][j] +
   matriz[i][1]*matriz[1][j] + matriz[i][2]*matriz[2][j];
            }
        }
```

```

        // Se imprime el resultado, garantizando la exclusion mutua
        lock(micerrojo);                    // Protoc. Entr.
        imprimir("A x A \n");
        for (int a = 0; a < 3; a++) {
            imprimir(matriz[a][0] + " " + matriz[a][1] + " " + matriz[a][2]);
        }
        imprimir("x");
        for (int a = 0; a < 3; a++) {
            imprimir(matriz[a][0] + " " + matriz[a][1] + " " + matriz[a][2]);
        }
        imprimir(" ");
        imprimir("A2");
        for (int a = 0; a < 3; a++) {
            imprimir(matrizRes[a][0] + " " + matrizRes[a][1] + " " + matrizRes[a][2]);
        }
        imprimir("\n");
        unlock(micerrojo);                  // Protoc. Sal.
    }
```

Listing 2: Pseudocódigo HiloSuma

```
HiloSuma(micerrojo)

    for (int reps = 0; reps < 10; reps++) {
        int[][] matriz = new int[3][3];
        int[][] matrizRes = new int[3][3];

        // Rellenar matriz aleatoria
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                matriz[i][j] = rand.nextInt(100);
            }
        }

        // Hacemos la suma
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                matrizRes[i][j] = matriz[i][j] + matriz[i][j];
            }
        }

        // Se imprime el resultado, garantizando la exclusion mutua
        lock(micerrojo);                    // Protoc. Entr.
        imprimir("A + A \n");
        for (int a = 0; a < 3; a++) {
            imprimir(matriz[a][0] + " " + matriz[a][1] + " " + matriz[a][2]);
        }
```

```
        imprimir("+");
        for (int a = 0; a < 3; a++) {
            imprimir(matriz[a][0] + " " + matriz[a][1] + " " +
    matriz[a][2]);
        }
        imprimir(" ");
        imprimir("2A");
        for (int a = 0; a < 3; a++) {
            imprimir(matrizRes[a][0] + " " + matrizRes[a][1] + " "
    + matrizRes[a][2]);
        }
        imprimir("\n");
        unlock(micerrojo);                  // Protoc. Sal.
     }
```

1.4. Cuestiones

¿Qué ocurriría si no se utilizase sincronización?

Sin sincronización, las líneas de los dos hilos se mezclarían en la consola. Como cada hilo hace varias llamadas a `println` seguidas, el sistema operativo puede interrumpirle entre una y otra para ejecutar el otro hilo. El resultado sería algo como esto:

Listing 3: Ejemplo de salida intercalada (sin cerrojo)

```
A  x  A
A  +  A
a1  a2  a3        a1  a2  a3        A2
a1  a2  a3        a1  a2  a3        2A
a4  a5  a6    x   a4  a5  a6    =   r1  r2  r3
```

Es decir, las líneas de `HiloMult` y `HiloSuma` aparecen entremezcladas y la salida es ilegible.

1.5. Código Java

A continuación se muestra el código programado para cada una de las clases del ejercicio:

Listing 4: Programa1.java

```java
package Ejercicio1;

import java.util.concurrent.locks.ReentrantLock;

/**
 * Programa principal del Ejercicio 1.
 * Lanza un hilo de multiplicacion y otro de suma usando un mismo
   cerrojo.
 */
public class Programa1 {


    public static void main(String[] args) {
        ReentrantLock cerrojo = new ReentrantLock();

        HiloMult mult = new HiloMult(cerrojo);
        HiloSuma sum = new HiloSuma(cerrojo);
        mult.start();
        sum.start();

    }

}
```

Listing 5: HiloSuma.java

```java
package Ejercicio1;

import java.util.Random;
```

```java
import java.util.concurrent.locks.ReentrantLock;

/**
 * Hilo que genera una matriz 3x3, calcula A + A y la muestra por pantalla.
 * La salida se protege con un cerrojo para evitar mezclas en consola.
 */
public class HiloSuma extends Thread {

     ReentrantLock micerrojo;
     Random rand = new Random();


    /**
     * Crea el hilo de suma con el cerrojo compartido.
     *
     * @param cerrojo cerrojo usado para proteger la salida por pantalla
     */
    public HiloSuma (ReentrantLock cerrojo) {
        super();
        this.micerrojo = cerrojo;
    }

    /**
     * Ejecuta 10 repeticiones: genera la matriz, calcula 2A e imprime el resultado.
     */
    @Override
    public void run() {
        super.run();
        for(int reps = 0; reps < 10; reps++) {
        int[][] matriz = new int[3][3];
        int[][] matrizRes = new int[3][3];

        for(int i = 0; i < 3; i++) {
            for(int j = 0; j<3; j++) {
                matriz[i][j] = rand.nextInt(100);
            }
        }

        for(int i = 0; i < 3; i++) {
            for(int j = 0; j<3; j++) {
                matrizRes[i][j] = matriz[i][j]+matriz[i][j];

            }
        }
        micerrojo.lock();
        try {
            System.out.println("A + A \n");
```

```
            for(int a = 0; a<3; a++) {
                    System.out.println(matriz[a][0]+ " " + matriz[a][1] + " " + matriz[a][2]);
            }
            System.out.println("+");

            for(int a = 0; a<3; a++) {
                System.out.println(matriz[a][0] + " " + matriz[a][1] + " "+ matriz[a][2]);
        }
            System.out.println(" ");
            System.out.println("2A");
            for(int a = 0; a<3; a++) {
                System.out.println(matrizRes[a][0] + " " + matrizRes[a][1] + " "+ matrizRes[a][2]);
        }
            System.out.println("\n");


        }

             finally {
                micerrojo.unlock();
            }

        }

    }









}
```

Listing 6: HiloMult.java

```
package Ejercicio1;

import java.util.Random;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Hilo que genera una matriz 3x3, calcula A x A y la muestra por pantalla.
 * La salida se protege con un cerrojo para que no se mezcle con otros hilos.
 */
public class HiloMult extends Thread {
```

```java

    ReentrantLock micerrojo;
    Random rand = new Random();

   /**
    * Crea el hilo de multiplicacion con el cerrojo compartido.
    *
    * @param cerrojo cerrojo usado para proteger la salida por pantalla
    */
   public HiloMult(ReentrantLock cerrojo) {
       super();
       this.micerrojo = cerrojo;
   }

   /**
    * Ejecuta 10 repeticiones: genera la matriz, calcula A2 e imprime el resultado.
    */
   @Override
   public void run() {
       super.run();
       for(int reps = 0; reps < 10; reps++) {

       int[][] matriz = new int[3][3];
       int[][] matrizRes = new int[3][3];

       for(int i = 0; i < 3; i++) {
           for(int j = 0; j<3; j++) {
               matriz[i][j] = rand.nextInt(100);
           }
       }

       for(int i = 0; i < 3; i++) {
           for(int j = 0; j<3; j++) {
               matrizRes[i][j] = matriz[i][0]*matriz[0][j]+matriz[i][1]*matriz[1][j] +  matriz[i][2]*matriz[2][j];

           }
       }
       micerrojo.lock();
       try {
           System.out.println("A x A \n");
           for(int a = 0; a<3; a++) {
                   System.out.println(matriz[a][0]+ " " + matriz[a][1] + " " + matriz[a][2]);
           }

           System.out.println("x");

           for(int a = 0; a<3; a++) {
```

```
                System.out.println(matriz[a][0] + " " + matriz[a][1] + " "+ matriz[a][2]);
        }
            System.out.println(" ");

            System.out.println("A2");
            for(int a = 0; a<3; a++) {
                System.out.println(matrizRes[a][0] + " " + matrizRes[a][1] + " "+ matrizRes[a][2]);
        }
            System.out.println("\n");


        }

             finally {
                micerrojo.unlock();
            }
        }


    }





}
```

2. Ejercicio 2 — Semáforos

2.1. Recursos no compartibles

Los recursos que no pueden ser usados por dos hilos a la vez en este ejercicio son:

- **Los 3 paneles físicos**: Cada panel es un recurso hardware de escritura. Dos hilos no pueden escribir en el mismo panel al mismo tiempo sin que la salida se corrompa, así que cada panel individual requiere acceso exclusivo.
- **El array** `libres`: Es la variable compartida (recurso software) que guarda qué paneles están ocupados o libres. Si dos hilos lo leen y modifican a la vez sin protección, pueden acabar asignándose el mismo panel.

2.2. Condiciones de sincronización

Sí existe una condición de sincronización en este problema. Un equipo no puede simplemente ir a buscar un panel libre en cualquier momento; primero tiene que esperar a que haya al menos uno disponible. Es decir, un hilo tiene que esperar a que otro hilo (equipo) termine de usar su panel y lo libere. Eso es exactamente una condición de sincronización: un proceso espera a que el estado del sistema cambie por acción de otro proceso.

Esta condición se implementa con el semáforo general `disponibles`, inicializado a 3 (el número de paneles). Cada vez que un equipo quiere usar un panel hace `wait(disponibles)`, lo que le bloquea si ya hay 3 equipos usando paneles. Cuando termina, hace `signal(disponibles)` para avisar al siguiente que está esperando.

Además, el acceso al array `libres` para buscar y marcar qué panel se va a usar (y luego para liberarlo) se protege con **exclusión mutua** mediante el semáforo binario `mutex`, para que dos hilos no modifiquen el array a la vez.

2.3. Pseudocódigo comentado

Listing 7: Pseudocódigo HiloEquipo

```
HiloEquipo(idequipo)
    int aacceder = 0;

    for (int reps = 0; reps < 3; reps++) {
        int[][] deportista1 = new int[5][5];
        int[][] deportista2 = new int[5][5];
        int[][] deportista3 = new int[5][5];
        int[][] deportista4 = new int[5][5];
        int[][][] deportistas = {deportista1, deportista2,
   deportista3, deportista4};
        int[][] R = new int[4][5];

        // Generar resultados para todos los deportistas del equipo
        for (int a = 0; a < 4; a++) {
```

```
            for (int i = 0; i < 5; i++) {
                deportistas[a][0][i] = rand.nextInt(61) + 90;
                deportistas[a][1][i] = rand.nextInt(401) + 400;
                deportistas[a][2][i] = rand.nextInt(71) + 10;
                deportistas[a][3][i] = rand.nextInt(301) + 300;
                deportistas[a][4][i] = rand.nextInt(151) + 150;
            }
        }

        // Calcular limites y pasarlos a matriz R
        for (int i = 0; i < 4; i++) {
            R[i][0] = calculaMin(deportistas[i][0]);
            R[i][1] = calculaMax(deportistas[i][1]);
            R[i][2] = calculaMax(deportistas[i][2]);
            R[i][3] = calculaMin(deportistas[i][3]);
            R[i][4] = calculaMin(deportistas[i][4]);
        }

        // Sincronizacion por los paneles: solo pasan 3 a la vez
        wait(disponibles);                      // Sincronizacion

        // Exclusion mutua para acceder al array de booleanos
        wait(mutex);                            // Protoc. Entr.
        for (int i = 0; i < 3; i++) {
            if (libres[i]) {
                aacceder = i;
                libres[i] = false;
                break;
            }
        }
        signal(mutex);                          // Protoc. Sal.

        // Fuera de la zona mutex escribimos en el panel
        String textoR = generarStringMatriz(R);
        paneles[aacceder].escribir_mensaje("Usando panel " + aacceder + " equipo " + idequipo + "\n");
        paneles[aacceder].escribir_mensaje("Matriz R (equipo " + idequipo + ") =\n" + textoR);
        paneles[aacceder].escribir_mensaje("Terminando panel " + aacceder + " equipo " + idequipo + "\n\n");

        // Exclusion mutua para actualizar disponibilidad del hueco tomado
        wait(mutex);                            // Protoc. Entr.
        libres[aacceder] = true;
        signal(mutex);                          // Protoc. Sal.

        // Avisa al siguiente que pueda estar esperando
        signal(disponibles);                    // Sincronizacion
    }
```

2.4. Cuestiones

a) Si consideramos un panel solamente ¿cómo es el acceso al mismo? Si consideramos los tres paneles ¿cómo es el acceso? Explica ambas situaciones.

- **Un solo panel**: El acceso es en **exclusión mutua**. Un panel es un recurso que solo puede usar un equipo a la vez; mientras un hilo está escribiendo en él, ningún otro puede hacerlo.
- **Los tres paneles**: Pueden escribir hasta 3 equipos a la vez, uno en cada panel. Es decir, el acceso es concurrente: varios hilos operan simultáneamente, pero nunca dos sobre el mismo panel.

b) ¿Qué tipo de semáforos has usado para resolver el problema (binarios, generales)?

Se han usado los dos tipos:

- **Semáforo binario (`mutex`, inicializado a 1)**: Para proteger en exclusión mutua el acceso al array `libres`. Así dos hilos no pueden leerlo y modificarlo a la vez y asignarse el mismo panel.
- **Semáforo general (`disponibles`, inicializado a 3)**: Para controlar cuántos equipos pueden estar usando paneles a la vez. Si los 3 paneles están ocupados, el siguiente equipo se bloquea en `wait(disponibles)` hasta que alguno haga `signal(disponibles)`.

2.5. Código Java

Listing 8: Programa2.java

```
package Ejercicio2;

import java.util.concurrent.Semaphore;

/**
 * Programa principal del Ejercicio 2.
 * Inicializa paneles y lanza los 10 hilos de equipos.
 */
public class Programa2 {
    static Semaphore disponibles = new Semaphore(3);
    static Semaphore mutex = new Semaphore(1);
    static volatile boolean[] libres = {true, true, true};
    static Panel panel1 = new Panel("Panel 0", 100, 100);
    static Panel panel2 = new Panel("Panel 1", 550, 100);
    static Panel panel3 = new Panel("Panel 2", 1000, 100);
    static Panel[] paneles = {panel1,panel2,panel3};




    public static void main(String[] args) {



```

```
        Thread[] equipos = new Thread[10];

        for(int i = 0; i<10; i++) {
            equipos[i] = new HiloEquipo(i);
            equipos[i].start();
        }




    }

}
```

Listing 9: HiloEquipo.java

```
package Ejercicio2;

import java.util.Random;
import java.util.concurrent.Semaphore;

/**
 * Hilo que representa a un equipo.
 * En cada repeticion genera resultados, construye la matriz
   resumen R
 * y la muestra en un panel libre.
 */
public class HiloEquipo extends Thread {

    Semaphore disponibles;
    int idequipo;


    /**
     * Crea el hilo de un equipo.
     *
     * @param idequipo identificador del equipo
     */
    public HiloEquipo(int idequipo) {
        super();
        this.idequipo = idequipo;
    }

    /**
     * Ejecuta 3 ciclos de trabajo del equipo y usa semaforos para
   acceder
     * a un panel compartido.
     */
    @Override
    public void run() {
        int aacceder= 0;
```

```
        for(int reps = 0; reps <3; reps++) {
        int[][] deportista1 = new int[5][5];
        int[][] deportista2 = new int[5][5];
        int[][] deportista3 = new int[5][5];
        int[][] deportista4 = new int[5][5];
        int[][][] deportistas = {deportista1, deportista2,
    deportista3, deportista4};
        int[][] R = new int[4][5];
        Random rand = new Random();


    for(int a = 0; a < 4; a++) {
        for(int i = 0; i < 5; i++) {
            deportistas[a][0][i] = rand.nextInt(61)+90; //Decimas
    de s
            deportistas[a][1][i] = rand.nextInt(401)+400; // cm
            deportistas[a][2][i] = rand.nextInt(71)+10;  // metros
            deportistas[a][3][i] = rand.nextInt(301)+300; //
    segundos
            deportistas[a][4][i] = rand.nextInt(151)+150; //decimas
     de s
        }
    }


    for(int i = 0; i < 4; i++) {
        R[i][0] = calculaMin(deportistas[i][0]);
        R[i][1] = calculaMax(deportistas[i][1]);
        R[i][2] = calculaMax(deportistas[i][2]);
        R[i][3] = calculaMin(deportistas[i][3]);
        R[i][4] = calculaMin(deportistas[i][4]);
        }



    try {
        Programa2.disponibles.acquire();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
        try {
            Programa2.mutex.acquire();
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }


        for(int i = 0;  i < 3; i++) {
            if(Programa2.libres[i] == true) {
                aacceder = i;
```

```
                Programa2.libres[i] = false;
                break;
            }
        }
            Programa2.mutex.release();

            String textoR = generarStringMatriz(R);
            Programa2.paneles[aacceder].escribir_mensaje("Usando panel " + aacceder + " el hilo (equipo) " + idequipo + "\n");
            Programa2.paneles[aacceder].escribir_mensaje("Matriz R (resumen del equipo " + idequipo + ": mejores marcas) =\n" + textoR);
            Programa2.paneles[aacceder].escribir_mensaje("Terminando de usar panel " + aacceder + " el hilo (equipo) " + idequipo + "\n\n");


            try {
                Programa2.mutex.acquire();
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
            Programa2.libres[aacceder] = true;
            Programa2.mutex.release();

            Programa2.disponibles.release();
        }



            }



    /**
     * Devuelve el valor maximo de un array.
     *
```

```
     * @param array array de entrada
     * @return valor maximo
     */
    private int calculaMax(int[] array) {
        int aux = array[0];
        for(int i = 1; i < array.length; i++) {
            if(aux < array[i]) {
                aux = array[i];
            }
        }
        return aux;
    }

    /**
     * Devuelve el valor minimo de un array.
     *
     * @param array array de entrada
     * @return valor minimo
     */
    private int calculaMin(int[] array) {
        int aux = array[0];
        for(int i = 1; i < array.length; i++) {
            if(aux > array[i]) {
                aux = array[i];
            }
        }
        return aux;
    }
    /**
     * Convierte una matriz en texto para mostrarla por panel.
     *
     * @param matriz matriz a convertir
     * @return texto de la matriz con saltos de linea
     */
    private String generarStringMatriz(int[][] matriz) {
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < matriz.length; i++) {
            for (int j = 0; j< matriz[i].length; j++) {
                sb.append(matriz[i][j]).append("\t");
            }
            sb.append("\n");
        }

        return sb.toString();
    }



}
```

3. Ejercicio 3 — Monitores

3.1. Recursos no compartibles

En esta simulación, los recursos que no pueden usarse por varios clientes a la vez son:

- **Los 3 tornos de acceso**: Cada torno es un recurso que solo puede gestionar la entrada de un cliente a la vez. Dos clientes no pueden pasar por el mismo torno simultáneamente.
- **Cada zona de entrenamiento (las 4 zonas)**: En la implementación, cada zona se modela como un recurso que solo puede tener un cliente entrenando a la vez (`ocupado[zona]`). Aunque hay 5 máquinas físicas, la variable de control es booleana por zona, así que se trata como acceso exclusivo.
- **La bicicleta premium**: Es un recurso único dentro de la zona Cardio. Solo un cliente puede usarla a la vez (`biciPremiumOcupada`).
- **La pantalla** (`System.out`): Imprimir el informe de cada cliente son varias líneas seguidas. Sin protección, los mensajes de distintos clientes se mezclarían.

3.2. Condiciones de sincronización

Hay varias condiciones de sincronización, todas del mismo tipo: un cliente tiene que esperar a que otro cliente libere un recurso antes de poder usarlo.

- **Espera para entrar por un torno**: Si los tres tornos están ocupados, el cliente tiene que quedarse bloqueado hasta que otro cliente termine y libere el suyo. El cliente espera en la cola del torno que tenga menos gente. Esta condición se gestiona con `await()` en la variable de condición del torno elegido.
- **Espera para entrar en una zona**: Si la zona elegida ya está ocupada, el cliente tiene que esperar hasta que el cliente que está dentro salga y haga `signal()`. Cada zona tiene su propia cola de espera (`Condition`).
- **Espera para usar la bicicleta premium**: Si un cliente quiere la bicicleta y ya la tiene otro, se queda bloqueado en la cola exclusiva de la bici hasta que ese cliente la libere y haga `signal()` en esa condición.

3.3. Pseudocódigo comentado

Listing 10: Pseudocódigo MonitorZonas

```
monitor MonitorZonas {
    private static int TIEMPO_PROMEDIO = 60;
    private Condition[] zona;           // cola de espera por cada
   zona
    private Condition biciPremium;      // cola de espera de la
   bici premium
    private int[] colas = {0, 0, 0, 0};                  //
   clientes esperando por zona
```

```
    private boolean[] ocupado = {false, false, false, false};
    private boolean biciPremiumOcupada = false;

    // METODO 1: ENTRAR EN ZONA
    public void entrar(int idZona) {
        while (ocupado[idZona]) {
            colas[idZona]++;
            delay(zona[idZona]);          // espera a que se libere la zona
            colas[idZona]--;
        }
        ocupado[idZona] = true;
    }

    // METODO 2: PEDIR BICI PREMIUM (solo Cardio)
    public void pedirBiciPremium() {
        while (biciPremiumOcupada) {
            delay(biciPremium);           // espera a que se libere
        }
        biciPremiumOcupada = true;
    }

    // METODO 3: SALIR DE ZONA
    public void salir(int idZona, boolean usoBici) {
        if (idZona == 0 && usoBici) {
            biciPremiumOcupada = false;
            resume(biciPremium);          // despierta siguiente en cola de bici premium
        }
        ocupado[idZona] = false;
        resume(zona[idZona]);             // despierta siguiente en cola de esa zona
    }

    // METODO 4: OBTENER ESTIMACIONES DE ESPERA
    public int[] getEstimaciones() {
        int[] estimaciones;
        for (int i = 0; i < 4; i++) {
            if (ocupado[i]) estimaciones[i] = TIEMPO_PROMEDIO * colas[i];
            else estimaciones[i] = 0;
        }
        return estimaciones;
    }

    // METODO 5: ELEGIR ZONA
    public int elegirZona() {
        zonasLibres = [];
        for (int i = 0; i < 4; i++) {
            if (!ocupado[i]) zonasLibres.add(i);
        }
```

```
        if (!empty(zonasLibres))
            return aleatorio(zonasLibres);
        else
            return zonaConMenorCola();
    }
}
```

Listing 11: Pseudocódigo MonitorTorno

```
monitor MonitorTorno {
    private Condition[] colaTorno;                // cola de espera por cada torno
    private boolean[] ocupado = {false, false, false};
    private int[] enCola = {0, 0, 0};            // cuantos esperan por cada torno

    // METODO 1: pedir un torno
    public int entrar(int id) {
        // Buscar primer torno libre
        for (int i = 0; i < 3; i++) {
            if (!ocupado[i]) {
                ocupado[i] = true;
                return i;
            }
        }
        // Si no hay libre: elegir el de menor cola
        int tornoElegido = tornoConMenorCola();
        enCola[tornoElegido]++;
        while (ocupado[tornoElegido]) {
            delay(colaTorno[tornoElegido]);      // suelta monitor, espera
        }
        enCola[tornoElegido]--;
        ocupado[tornoElegido] = true;
        return tornoElegido;
    }

    // METODO 2: liberar un torno
    public void salir(int tornoId) {
        ocupado[tornoId] = false;
        resume(colaTorno[tornoId]);              // despierta al siguiente en esa cola
    }
}
```

Listing 12: Pseudocódigo HiloCliente

```
HiloCliente(id)
    int X = random(1..5);          // tiempo validacion torno
    int Y = random(...);           // tiempo entrenamiento (30-120 min, convertido a ms)

    // FASE 1: TORNO
    int miTorno = monitorTorno.entrar(id);   // dentro del monitor: asigna torno
    sleep(X);                                // FUERA del monitor: usa el torno
    monitorTorno.salir(miTorno);             // dentro del monitor: libera torno

```

```
    // FASE 2: ELEGIR ZONA + IMPRIMIR
    int[] estimaciones = monitorZonas.getEstimaciones();
    int zona = monitorZonas.elegirZona();

    // Imprimir ANTES de entrar (formato del enunciado)
    lock(consola);                                          //
Protoc. Entr.
    imprimir
("-------------------------------------------------------------")
;
    imprimir("Cliente " + id + " ha pasado por el torno: " +
miTorno);
    imprimir("Tiempo en el torno (acceso): " + X);
    imprimir("Zona elegida: " + zona);
    imprimir("Tiempo de entrenamiento: " + Y);
    imprimir("Estimacion de espera (sin incluirse a si mismo):");
    imprimir("  Zona1(Cardio)=" + estimaciones[0] + ", Zona2(Fuerza
)=" + estimaciones[1]
            + ", Zona3(Funcional)=" + estimaciones[2] + ",");
    imprimir("  Zona4(Estiramientos)=" + estimaciones[3]);
    if (zona == CARDIO) imprimir("Espera bicicleta premium (si
aplica)=0");
    imprimir
("-------------------------------------------------------------")
;
    unlock(consola);                                        //
Protoc. Sal.

    // FASE 3: ENTRAR EN ZONA
    monitorZonas.entrar(zona);              // si la zona esta
llena -> espera en cola

    // FASE 4: BICI PREMIUM (solo Cardio)
    boolean usoBici = false;
    if (zona == CARDIO && random() < 0.30) {
        monitorZonas.pedirBiciPremium();    // espera en cola
exclusiva si esta ocupada
        usoBici = true;
    }

    // FASE 5: ENTRENAR
    sleep(Y);                               // FUERA del monitor

    // FASE 6: SALIR
    monitorZonas.salir(zona, usoBici);      // libera zona,
despierta siguiente
```

3.4. Cuestiones

a) Indica si la acción de entrenar es concurrente o en exclusión mutua y justifica.

Entrenar es **concurrente**. El entrenamiento se simula con `Thread.sleep(Y)`, y esa llamada se hace **fuera** de cualquier método del monitor, es decir, el hilo ya ha liberado el cerrojo antes de ponerse a «dormir». Como en ese momento no se toca ningún recurso compartido, varios clientes pueden estar entrenando a la vez sin ningún problema.

b) ¿Qué tipo de monitor Java has usado? Justifica.

Se ha implementado el monitor con `ReentrantLock` y variables `Condition`, usando `await()` para bloquear y `signal()` para despertar, que son los equivalentes Java de `delay` y `resume` del pseudocódigo. La política de desbloqueo que aplica este mecanismo es **DC (Desbloquear y Continuar)**: cuando un hilo llama a `signal()`, el hilo despertado no puede ejecutarse de inmediato, sino que queda pendiente hasta que el hilo que señalizó termine su método y libere el cerrojo. Hemos elegido esta solución porque necesitamos varias variables de condición independientes: una por cada zona de entrenamiento y otra exclusiva para la bicicleta premium, ya que cada una gestiona una cola de espera distinta.

c) En el monitor, ¿has usado notify/signal o notifyAll/signalAll? Justifica.

Hemos usado `signal()`. Al tener una variable de condición distinta para cada recurso (cada zona y la bici premium), cuando hacemos `signal()` sobre la condición de una zona concreta solo se despierta al cliente que está esperando exactamente ese recurso. Con `signalAll()` despertaríamos a todos los hilos bloqueados, la mayoría comprobaría su condición, vería que no se cumple y volvería a bloquearse, lo cual no tiene ningún sentido y es ineficiente.

d) ¿Cómo se ha resuelto la exclusión mutua de la pantalla en este problema?

Con un `ReentrantLock consola` compartido entre todos los clientes. Antes de imprimir, el hilo hace `consola.lock()` y al terminar `consola.unlock()` dentro de un bloque `finally` para asegurarse de que siempre se libera (aunque falle algo). De este modo solo un cliente puede imprimir a la vez y las líneas de distintos clientes no se mezclan.

3.5. Código Java

Listing 13: Programa.java

```java
package Ejercicio3;

import java.util.concurrent.locks.ReentrantLock;

/**
 * Programa principal del Ejercicio 3.
 * Crea monitores compartidos y lanza 50 hilos cliente.
 */
public class Programa {

    public static void main(String[] args) {

        MonitorTorno monitorTorno = new MonitorTorno();
        MonitorZonas monitorZonas = new MonitorZonas();
        ReentrantLock consola = new ReentrantLock();

        for (int i = 0; i < 50; i++) {
```

```
            new HiloCliente(i, monitorTorno, monitorZonas, consola)
    .start();
        }
    }

}
```

Listing 14: MonitorZonas.java

```
package Ejercicio3;

import java.util.ArrayList;
import java.util.Random;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Monitor que gestiona entrada y salida en las zonas del gimnasio
 * y el uso de la bicicleta premium de la zona de cardio.
 */
public class MonitorZonas {

    private final static int TIEMPO_PROMEDIO = 9;
    private ReentrantLock cerrojo;
    private Condition[] zona;
    private Condition biciPremium;
    private int[] colas;
    private boolean[] ocupado;
    private boolean biciPremiumOcupada;
    private Random random;

    /**
     * Inicializa el monitor de zonas y sus colas.
     */
    public MonitorZonas() {
        this.cerrojo = new ReentrantLock();
        this.zona = new Condition[] {
            cerrojo.newCondition(),
            cerrojo.newCondition(),
            cerrojo.newCondition(),
            cerrojo.newCondition()
        };
        this.biciPremium = cerrojo.newCondition();
        this.colas = new int[] {0, 0, 0, 0};
        this.ocupado = new boolean[] {false, false, false, false};
        this.biciPremiumOcupada = false;
        this.random = new Random();
    }


    /**
     * Entra en una zona; espera si esta ocupada.
```

```
     *
     * @param idZona identificador de la zona
     */
    public void entrar(int idZona) {
        cerrojo.lock();
        try {
            while(ocupado[idZona]){
                colas[idZona]++;
                zona[idZona].await();
                colas[idZona]--;
            }


            ocupado[idZona] = true;
        }catch (InterruptedException e) {
            e.printStackTrace();
        }finally {
            cerrojo.unlock();
        }
    }

    /**
     * Solicita la bicicleta premium y espera si esta ocupada.
     */
    public void pedirBiciPremium() {
        cerrojo.lock();
        try {
            while(biciPremiumOcupada) {
                biciPremium.await();
            }
            biciPremiumOcupada = true;

        }catch (InterruptedException e) {
            e.printStackTrace();
        }finally {
            cerrojo.unlock();
        }
    }

    /**
     * Elige una zona libre al azar; si no hay libres, elige la de
menor cola.
     *
     * @return identificador de la zona elegida
     */
    public int elegirZona() {
        cerrojo.lock();
        int zonaElegida = -1;
        try {
            ArrayList<Integer> zonasLibres = new ArrayList<>();
            for(int i = 0; i < 4; i++) {
```

```
                if(!ocupado[i]) {
                    zonasLibres.add(i);
                }
            }

            if (!zonasLibres.isEmpty()) {
                return zonasLibres.get(random.nextInt(zonasLibres.size()));
            }else {
                zonaElegida = 0;
                for (int i = 1; i < 4; i++) {
                    if (colas[i] < colas[zonaElegida]) {
                        zonaElegida = i;
                    }
                }

                return zonaElegida;
            }

        }finally {
            cerrojo.unlock();
        }
    }


    /**
     * Devuelve una estimacion simple del tiempo de espera por zona.
     *
     * @return array con las 4 estimaciones
     */
    public int[] getEstimaciones(){
        cerrojo.lock();
        try {
            int[] estimaciones = new int[4];
            for(int i = 0; i < 4; i++) {
                if(ocupado[i])
                    estimaciones[i] = TIEMPO_PROMEDIO * colas[i];
                else
                    estimaciones[i] = 0;
            }
            return estimaciones;
        }finally {
            cerrojo.unlock();
        }
    }


    /**
     * Sale de la zona y, si aplica, libera la bicicleta premium.
     *
```

```
     * @param idZona identificador de la zona
     * @param usoBici indica si el cliente uso la bicicleta premium
     */
    public void salir(int idZona, boolean usoBici) {
        cerrojo.lock();
        try {
            if((idZona == 0) && (usoBici)) {
                biciPremiumOcupada = false;
                biciPremium.signal();
            }
            ocupado[idZona] = false;
            zona[idZona].signal();
        }finally {
            cerrojo.unlock();
        }

    }



}
```

Listing 15: MonitorTorno.java

```
package Ejercicio3;

import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Monitor que gestiona el acceso a los 3 tornos del gimnasio.
 */
public class MonitorTorno {

    private ReentrantLock cerrojo;
    private Condition[] colaTorno;
    private boolean[] ocupado;
    private int[] enCola;

    /**
     * Inicializa el estado del monitor de tornos.
     */
    public MonitorTorno() {
        this.cerrojo = new ReentrantLock();
        this.colaTorno = new Condition[] {
            cerrojo.newCondition(),
            cerrojo.newCondition(),
            cerrojo.newCondition()
        };
        this.ocupado = new boolean[] {false, false, false};
        this.enCola = new int[] {0, 0, 0};
    }


    /**
     * Asigna un torno al cliente.
     * Si no hay tornos libres, espera en la cola del torno con
   menor espera.
     *
     * @param id identificador del cliente
     * @return identificador del torno asignado
     */
    public int entrar(int id) {
        cerrojo.lock();
        int tornoElegido = -1;
        try {
            // 1. Buscar primer torno libre
            for (int i = 0; i < 3; i++) {
                if (!ocupado[i]) {
                    tornoElegido = i;
                    break;
                }
            }

```

```
            // 2. Si no hay libre elegir el de menor cola
            if (tornoElegido == -1) {
                tornoElegido = 0;
                for (int i = 1; i < 3; i++) {
                    if (enCola[i] < enCola[tornoElegido]) {
                        tornoElegido = i;
                    }
                }
                // 3. Esperar en la cola del torno elegido
                enCola[tornoElegido]++;
                while (ocupado[tornoElegido]) {
                    colaTorno[tornoElegido].await();
                }
                enCola[tornoElegido]--;
            }
            // 4. Reclamar el torno
            ocupado[tornoElegido] = true;
            return tornoElegido;

        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        } finally {
            cerrojo.unlock();
        }

    }

    /**
     * Libera un torno y despierta a un cliente que espera en su
   cola.
     *
     * @param tornoid identificador del torno a liberar
     */
    public void salir(int tornoid) {
        cerrojo.lock();
        try{
            ocupado[tornoid] = false;
            colaTorno[tornoid].signal();
        }finally {
            cerrojo.unlock();
        }
    }




}
```

Listing 16: HiloCliente.java

```
package Ejercicio3;

```

```
import java.util.Random;
import java.util.concurrent.locks.ReentrantLock;

/**
 * Hilo que simula el recorrido de un cliente por torno y zona de entrenamiento.
 */
public class HiloCliente extends Thread {

    private final static int CARDIO = 0;
    private int id;
    private ReentrantLock consola;
    private MonitorTorno monitorTorno;
    private MonitorZonas monitorZonas;

     /**
      * Crea un cliente con los monitores compartidos.
      *
      * @param id identificador del cliente
      * @param monitorTorno monitor para acceso a tornos
      * @param monitorZonas monitor para gestion de zonas
      * @param consola cerrojo para escribir por pantalla sin mezclar salida
      */
     public HiloCliente(int id, MonitorTorno monitorTorno, MonitorZonas monitorZonas,
                        ReentrantLock consola) {
         super();
         this.id = id;
         this.monitorTorno = monitorTorno;
         this.monitorZonas = monitorZonas;
         this.consola = consola;
     }

    /**
     * Ejecuta el flujo del cliente: torno, eleccion de zona, posible bici premium,
     * entrenamiento y salida.
     */
    @Override
    public void run(){
        Random random = new Random();
        int X = (random.nextInt(5) + 1) * 1000;      // 1-5 segundos en torno
        int Y = (random.nextInt(91) + 30) * 100;      // 3-12 segundos entrenando


        //FASE 1: TORNO
        int miTorno;
        try {
```

```
            miTorno = monitorTorno.entrar(id);
            Thread.sleep(X);
            monitorTorno.salir(miTorno);

        //FASE 2: Elejir zona + imprimir
        int zona = monitorZonas.elegirZona(); //Elegimos zona

        int estimaciones[] = monitorZonas.getEstimaciones();

        consola.lock();
        try {
        System.out.println
    ("-------------------------------------------------------------")
    ;
        System.out.println("Cliente " + id + " ha pasado por el
    torno: " + miTorno);
        System.out.println("Tiempo en el torno (acceso): " + X);
        System.out.println("Zona elegida: " + zona);
        System.out.println("Tiempo de entrenamiento: " + Y);
        System.out.println("Estimacion de espera (sin incluirse a
    si mismo):");
        System.out.println("  Zona1(Cardio)=" + estimaciones[0] +
    ", Zona2(Fuerza)=" + estimaciones[1]
                + ", Zona3(Funcional)=" + estimaciones[2] + ",");
        System.out.println("  Zona4(Estiramientos)=" + estimaciones
    [3]);
        if (zona == CARDIO) System.out.println("Espera bicicleta
    premium (si aplica)=0");
        System.out.println
    ("-------------------------------------------------------------")
    ;
        } finally {
            consola.unlock();
        }


        //FASE 3: Entrar en zona
        monitorZonas.entrar(zona);

        //FASE 4: Bicicleta premium
        boolean usoBici = false;
        if (zona == CARDIO && random.nextDouble() < 0.30){
            monitorZonas.pedirBiciPremium();
            usoBici = true;

        }

        //FASE 5: Entrenar
        Thread.sleep(Y);

        //FASE 6: Salir
```

```
        monitorZonas.salir(zona, usoBici);

        } catch (InterruptedException e) {
            e.printStackTrace();
        }

    }



}
```

4. Ejercicio 4 — Paso de Mensajes Asíncrono

4.1. Recursos no compartibles

En este ejercicio los recursos que no se pueden usar a la vez son:

- **El torno R y el torno L**: Cada torno solo puede atender a un aficionado al mismo tiempo. Si dos aficionados intentaran usar el mismo torno a la vez, habría conflicto. Son dos recursos independientes, así que sí se pueden usar en paralelo entre sí (uno en R y otro en L al mismo tiempo), pero no dos en el mismo.
- **La pantalla** (`System.out`): Imprimir el informe de cada aficionado son varias líneas. Sin protección, los mensajes se mezclan.

4.2. Condiciones de sincronización

Hay dos condiciones de sincronización en este ejercicio:

- **Espera para usar el torno asignado**: Tras recibir del controlador qué torno le corresponde, el aficionado envía una petición al buzón de ese torno (`peticionTornoR` o `peticionTornoL`) y luego hace `receive()` en su buzón privado esperando confirmación. Si el torno está ocupado, el controlador no atiende ese buzón (porque la guarda está cerrada) y el aficionado se queda bloqueado hasta que el torno se libere y el controlador le responda. Un aficionado espera a que otro aficionado libere el torno antes de poder usarlo.
- **Exclusión mutua de la pantalla**: Un aficionado tiene que esperar a que otro termine de imprimir antes de poder imprimir él. Esto se implementa con el buzón `mutexPantalla`: al principio hay un mensaje dentro (el testigo). Para imprimir, el aficionado hace `receive(mutexPantalla` que le bloquea si otro ya tomó el testigo. Al terminar, hace `send(mutexPantalla)` para devolver el testigo y que el siguiente pueda pasar. Es la forma de hacer exclusión mutua con paso de mensajes.

4.3. Pseudocódigo comentado

Listing 17: Pseudocódigo HiloControlador

```
HiloControlador()
    boolean tornoRLibre = true
    boolean tornoLLibre = true
    int liberacionesProcesadas = 0
    int TOTAL_VALIDACIONES = 250 // (50 * 5)

    // Inicializar el mutex de la pantalla metiendo el testigo en
   el buzon
    send(mutexPantalla, "llavePantalla")

    while (true) {
        select {
```

```
            // Asignacion de torno inicial y tiempo estimado (siempre abierto)
            receive(buzonPeticiones, id);
            int tEstimado = rand.nextInt(10) + 1;
            if (tEstimado > 5) torno = "L";
            else torno = "R";
            respuesta = new RespuestaControlador(torno, tEstimado);
            send(buzonRespuestas[id], respuesta);
            or
            // Peticion para usar Torno R (guarda: torno R libre)
            when tornoRLibre =>
            receive(peticionTornoR, id);
            tornoRLibre = false;
            send(buzonRespuestas[id], "llave");
            or
            // Peticion para usar Torno L (guarda: torno L libre)
            when tornoLLibre =>
            receive(peticionTornoL, id);
            tornoLLibre = false;
            send(buzonRespuestas[id], "llave");
            or
            // Liberacion de un torno (guarda: al menos un torno ocupado)
            when !tornoRLibre || !tornoLLibre =>
            receive(buzonLiberacion, torno);
            liberacionesProcesadas++;
            if (torno == "R") tornoRLibre = true;
            else tornoLLibre = true;
            if (liberacionesProcesadas >= TOTAL_VALIDACIONES) return;
        }
    }
```

Listing 18: Pseudocódigo HiloAficionado

```
HiloAficionado(id)

    for (int i = 0; i < 5; i++) {
        sleep(rand.nextInt(500) + 50);

        // Enviar peticion al controlador en el buzon de peticiones general
        send(buzonPeticiones, id);

        // Bloquear hasta recibir respuesta indicando torno asignado y tiempo
        receive(miBuzon, respuesta);

        // Comunicar intencion de acceso al torno especifico ("R" o "L")
        if (respuesta.torno == "R") send(peticionTornoR, id);
        else send(peticionTornoL, id);
```

```

        // Esperar a que el controlador confirme que el torno esta disponible
        receive(miBuzon, llave);

        // Tiempo de validacion/uso del torno
        sleep(respuesta.tiempoEstimado * 100);

        // Avisar al controlador que el torno utilizado ha sido liberado
        send(buzonLiberacion, respuesta.torno);

        // Exclusion mutua de pantalla mediante testigo en buzon
        receive(mutexPantalla, llavePantalla);  // tomar testigo (bloquea si otro lo tiene)
        imprimir("Aficionado " + id + " ha usado la cola " + respuesta.torno);
        imprimir("Tiempo de validacion = " + respuesta.tiempoEstimado);
        imprimir("Thread.sleep(" + respuesta.tiempoEstimado + ")");
        imprimir("Aficionado " + id + " liberando la cola " + respuesta.torno);
        send(mutexPantalla, llavePantalla);     // devolver testigo
    }
```

4.4. Cuestiones

a) ¿Se pueden usar simultáneamente los dos tornos? Justifica la respuesta.

Sí. El controlador tiene dos variables independientes (`tornoRLibre` y `tornoLLibre`) y usa guardas en el `select` para cada buzón por separado. Si el torno R está ocupado, su guarda se cierra y el controlador deja de atender peticiones para R, pero la guarda de L sigue abierta. Así puede haber un aficionado en R y otro en L al mismo tiempo sin ningún problema.

b) ¿Cómo has resuelto la exclusión mutua de la pantalla?

Con un buzón dedicado (`mutexPantalla`) que actúa como testigo (token). Al arrancar, el controlador deposita el testigo en ese buzón. Antes de imprimir, cada aficionado hace `receive(mutexPantalla)`: si el buzón está vacío porque otro aficionado ya tomó el testigo, se bloquea hasta que ese otro haga `send(mutexPantalla)` y lo devuelva. De este modo solo un aficionado puede estar imprimiendo a la vez.

4.5. Código Java

Listing 19: Programa4.java

```
package Ejercicio4;

import messagepassing.MailBox;

```

```java
/**
 * Programa principal del Ejercicio 4.
 * Crea los buzones, lanza el controlador y los 50 hilos de aficionados.
 */
public class Programa4 {


    public static void main(String[] args) {
        int NUM_AFICIONADOS = 50;

        MailBox buzonPeticiones = new MailBox();
        MailBox buzonLiberacion = new MailBox();
        MailBox mutexPantalla = new MailBox();
        MailBox peticionTornoR = new MailBox();
        MailBox peticionTornoL = new MailBox();



        MailBox[] buzonesAficionados = new MailBox[NUM_AFICIONADOS];
        for(int i = 0; i < NUM_AFICIONADOS; i++) {
            buzonesAficionados[i] = new MailBox();
        }
        Thread controlador = new HiloControlador(buzonPeticiones, buzonesAficionados, buzonLiberacion, mutexPantalla, peticionTornoR, peticionTornoL);
        controlador.start();
        Thread[] aficionados = new Thread[NUM_AFICIONADOS];
        for(int i = 0; i < NUM_AFICIONADOS; i++) {
            aficionados[i] = new HiloAficionado(i, buzonPeticiones, buzonesAficionados[i], buzonLiberacion, mutexPantalla, peticionTornoR, peticionTornoL);
            aficionados[i].start();
        }

    }
}
```

Listing 20: HiloControlador.java

```java
package Ejercicio4;

import messagepassing.*;

import java.util.Random;

import messagepassing.MailBox;

/**
 * Hilo controlador que asigna torno, concede acceso y procesa liberaciones.
```

```
 */
public class HiloControlador extends Thread{
    private MailBox buzonPeticiones;
    private MailBox[] buzonRespuestas;
    private MailBox buzonLiberacion;
    private MailBox mutexPantalla;
    private Selector sel;
    private MailBox peticionTornoR;
    private MailBox peticionTornoL;
    private boolean tornoRLibre;
    private boolean tornoLLibre;
    private Random rand;
    private int liberacionesProcesadas;
    private static final int TOTAL_VALIDACIONES = 50 * 5;



    /**
     * Crea el controlador con todos los buzones compartidos.
     *
     * @param buzonPeticiones buzon de peticiones de aficionados
     * @param buzonRespuestas buzones de respuesta por aficionado
     * @param buzonLiberacion buzon para liberar torno
     * @param mutexPantalla buzon usado como mutex de pantalla
     * @param peticionTornoR buzon de peticiones del torno R
     * @param peticionTornoL buzon de peticiones del torno L
     */
    public HiloControlador(MailBox buzonPeticiones, MailBox[]
   buzonRespuestas, MailBox buzonLiberacion,MailBox mutexPantalla,
   MailBox peticionTornoR,  MailBox peticionTornoL) {
        super();
        this.buzonPeticiones = buzonPeticiones;
        this.buzonRespuestas = buzonRespuestas;
        this.buzonLiberacion = buzonLiberacion;
        this.mutexPantalla = mutexPantalla;
        this.peticionTornoL = peticionTornoL;
        this.peticionTornoR = peticionTornoR;

        this.rand = new Random();
        this.tornoRLibre = true;
        this.tornoLLibre = true;
        this.liberacionesProcesadas = 0;


        this.sel = new Selector();
        sel.addSelectable(buzonPeticiones, false);
        sel.addSelectable(peticionTornoR, false);
        sel.addSelectable(peticionTornoL, false);
        sel.addSelectable(buzonLiberacion, false);


```

```
    }
    /**
     * Atiende peticiones de asignacion, entradas a tornos y liberaciones
     * hasta completar todas las validaciones previstas.
     */
    @Override
    public void run() {

        mutexPantalla.send("llavePantalla");
        while(true) {

        buzonPeticiones.setGuardValue(true);
        peticionTornoR.setGuardValue(tornoRLibre);
        peticionTornoL.setGuardValue(tornoLLibre);
        buzonLiberacion.setGuardValue(!tornoRLibre || !tornoLLibre);

            switch (sel.selectOrBlock()) {
            case 1: {
                int id = (int) buzonPeticiones.receive();
                int tEstimado = rand.nextInt(10)+1;
                String torno;
                if(tEstimado > 5) {
                    torno = "L";
                }
                else {
                    torno = "R";
                }
                RespuestaControlador respuesta = new RespuestaControlador(torno, tEstimado);
                buzonRespuestas[id].send(respuesta);
                break;

            }
            case 2: {
                int id = (int) peticionTornoR.receive();
                tornoRLibre = false;
                buzonRespuestas[id].send("llave");
                break;
            }
            case 3:{
                int id = (int) peticionTornoL.receive();
                tornoLLibre = false;
                buzonRespuestas[id].send("llave");
                break;
            }

```

```


            case 4: {

                String torno = (String) buzonLiberacion.receive();
                liberacionesProcesadas++;
                if(torno.equals("R")) {
                    tornoRLibre = true;
            }
                else {
                    tornoLLibre = true;
                }
                if (liberacionesProcesadas >= TOTAL_VALIDACIONES) {
                    return;
                }
                break;
            }
            }
        }



        }
    }
```

Listing 21: HiloAficionado.java

```
package Ejercicio4;

import java.util.Random;

import messagepassing.MailBox;

/**
 * Hilo que representa a un aficionado.
 * En cada vuelta solicita torno, valida entrada y libera el torno.
 */
public class HiloAficionado extends Thread {

    private final int id;
    private MailBox buzonPeticiones;
    private MailBox miBuzon;
    private MailBox peticionTornoR;
    private MailBox peticionTornoL;
    private MailBox buzonLiberacion;
    private MailBox mutexPantalla;
    private Random rand;

```

```


    /**
     * Crea un aficionado y sus buzones de comunicacion.
     *
     * @param id identificador del aficionado
     * @param buzonPeticiones buzon para pedir asignacion de torno
     * @param miBuzon buzon personal para recibir respuesta del controlador
     * @param buzonLiberacion buzon para indicar liberacion de torno
     * @param mutexPantalla buzon usado como exclusion mutua de pantalla
     * @param peticionTornoR buzon de peticiones del torno R
     * @param peticionTornoL buzon de peticiones del torno L
     */
    public HiloAficionado(int id, MailBox buzonPeticiones, MailBox miBuzon,MailBox buzonLiberacion, MailBox mutexPantalla, MailBox peticionTornoR, MailBox peticionTornoL) {
        super();
        this.id = id;
        this.buzonPeticiones = buzonPeticiones;
        this.miBuzon = miBuzon;
        this.buzonLiberacion = buzonLiberacion;
        this.mutexPantalla = mutexPantalla;
        this.peticionTornoL = peticionTornoL;
        this.peticionTornoR = peticionTornoR;
        this.rand = new Random();

    }



    /**
     * Ejecuta 5 ciclos de salida y reentrada, coordinados por paso de mensajes.
     */
    @Override
    public void run() {
        try {
        for(int i = 0; i<5; i++) {
            Thread.sleep(rand.nextInt(500)+50);
            buzonPeticiones.send(id);
            RespuestaControlador respuesta;
            respuesta = (RespuestaControlador) miBuzon.receive();
            if(respuesta.getTorno().equals("R")) {
                peticionTornoR.send(id);
            }
            else {
                peticionTornoL.send(id);
            }
```

```
            miBuzon.receive();
            Thread.sleep(respuesta.getTiempoEstimado()*100);
            buzonLiberacion.send(respuesta.getTorno());

            String llavePantalla = (String) mutexPantalla.receive();
            System.out.println("Aficionado " + this.id + " ha usado la cola " + respuesta.getTorno());
            System.out.println("Tiempo de validación = " + respuesta.getTiempoEstimado());
            System.out.println("Thread.sleep(" + respuesta.getTiempoEstimado() + ")");
            System.out.println("Aficionado " + this.id + " liberando la cola " + respuesta.getTorno()+ "\n");
            mutexPantalla.send(llavePantalla);

        }
    }
        catch (InterruptedException e) {
            e.printStackTrace();
            }

    }

}
```

Listing 22: RespuestaControlador.java

```
package Ejercicio4;

import java.io.Serializable;

/**
 * Mensaje de respuesta del controlador con torno asignado y tiempo estimado.
 */
public class RespuestaControlador implements Serializable {

private static final long serialVersionUID = 1L;
private String torno;
private int tiempoEstimado;

    /**
     * Crea una respuesta del controlador.
     *
     * @param torno torno asignado (R o L)
     * @param tiempoEstimado tiempo estimado de validacion
     */
public RespuestaControlador(String torno, int tiempoEstimado) {
    super();
    this.torno = torno;
    this.tiempoEstimado = tiempoEstimado;
```

```
}

/**
 * Devuelve el torno asignado.
 *
 * @return torno asignado
 */
public String getTorno() {
    return torno;
}

/**
 * Devuelve el tiempo estimado de validacion.
 *
 * @return tiempo estimado
 */
public int getTiempoEstimado() {
    return tiempoEstimado;
}




}
```